CONTENTS

1. TITLE OF THE PROJECT **1**

2. INTRODUCTION AND OBJECTIVES **1**

2.1 Introduction 1

2.2 Problem Statement 2

2.3 Objectives of the Project 2

2.4 Scope of the Project 2

2.5 Intended Users 2

3. PROJECT CATEGORY **2**

4. ANALYSIS **3**

4.1 Existing System and its Limitations 3

4.2 Proposed System 3

4.3 Feasibility Study 4

4.4 Software Requirements Specification (SRS) 4

4.5 System Architecture 6

4.6 Use Case Diagram 7

4.7 Data Flow Diagrams 9

4.8 Entity-Relationship Diagram 16

4.9 Class Diagram 18

4.10 Data Dictionary 18

5. COMPLETE STRUCTURE OF THE PROJECT **20**

5.1 Modules and their Description 20

5.2 Data Structures 22

5.3 Database Design 22

5.4 Process Logic for Each Module 23

5.5 Testing Details 27

5.6 Reports Generation 29

5.7 Security Implementation 29

6. TOOLS / PLATFORM, HARDWARE AND SOFTWARE REQUIREMENTS **30**

6.1 Tools and Technologies 30

6.2 Software Requirements 30

6.3 Hardware Requirements 30

7. INDUSTRY / CLIENT **30**

8. FUTURE SCOPE AND FURTHER ENHANCEMENT **31**

8.1 Limitations of the Present Version 31

8.2 Future Scope 31

9. BIBLIOGRAPHY **31**

10. DECLARATION **32**

LIST OF FIGURES

Figure 1: Architecture of Hnazer 7
Figure 2: Use case diagram 8
Figure 3: Level 0 DFD (context diagram) 10
Figure 4: Level 1 DFD 11
Figure 5: Level 2 DFD for Process 1.0 (Authentication) 13
Figure 6: Level 2 DFD for Process 2.0 (Daily log) 14
Figure 7: Level 2 DFD for Process 3.0 (Scores and risk) 15
Figure 8: Level 2 DFD for Process 5.0 (AI advisor) 16
Figure 9: Entity-Relationship diagram 16
Figure 10: Class diagram 18
Figure 11: Module hierarchy 21
Figure 12: Page navigation of the system 23
Figure 13: Scoring and burnout-risk process 25
Figure 14: Overall workflow of the system 27

1. TITLE OF THE PROJECT

HNAZER

Lifestyle Journal and Self-Analysis Tracker

Item	Details
Project title	Hnazer – Lifestyle Journal and Self-Analysis Tracker
Project category	Web application; RDBMS (Database Management System); Information Security (local accounts and sessions); Applied AI (rule-based analysis with an optional Gemini advisor)
Front end	React 18, Vite, Tailwind CSS (with PostCSS and Autoprefixer), Chart.js
Back end	Node.js 22.5 or later local service (server/index.js) with a JSON API, bearer-token sessions and scrypt password hashing
Database	SQLite accessed through the built-in node:sqlite module, with three related tables and foreign keys
AI service	Built-in rule-based advisor (runs on the device); optional Google Gemini API, used only on explicit choice
Output	Dashboard cards, trend charts, radar and heatmap, burnout-risk summary, history table, CSV export, JSON backup, printable report
Type of project	Individual academic project (BCSP-064, 8 credits)
Industry / client	Not applicable (see Section 7)
Keywords	Lifestyle journal, wellness score, burnout risk, local-first, SQLite, React, Node.js, Gemini

2. INTRODUCTION AND OBJECTIVES

2.1 Introduction

Many people want to understand how their daily habits affect how they feel, but the relevant information is scattered. Sleep hours are noted in one place and workouts in another, while water intake and screen time are rarely recorded at all, and mood and diet are only remembered vaguely. Without a single record it is hard to notice patterns, such as short sleep combined with long screen time, or to see a slide towards burnout before it happens.

Paper notebooks and spreadsheets can store such entries, but they need manual calculation and give no instant feedback. Many commercial health and fitness applications, on the other hand, depend on wearable sensors, ask for e-mail accounts or cloud sign-in, and keep personal data on third-party servers.

Hnazer is a local-first lifestyle journaling and self-analysis web application. A user creates a local account (username, display name and password; no e-mail address is needed) and then records one daily entry covering sleep, exercise, water intake, screen time, mood, diet and optional notes. The system stores the profile and logs in an SQLite database on the user’s own computer, calculates a rule-based score for every metric, combines them into a weighted wellness score, and estimates a burnout risk from recent records with alerts and recommendations. The dashboard presents metric cards, trend charts, a mood distribution, a six-axis radar and a 35-day heatmap. A built-in advisor works entirely on the device, while an optional Gemini advisor can be chosen explicitly for an external AI analysis. JSON and CSV export, a printable report, a reference library, a light/dark theme and editable daily targets complete the system.

Hnazer does not connect to biometric sensors and does not give medical diagnoses. All values are entered by the user, and every score, label and recommendation is an informational heuristic.

2.2 Problem Statement

Individuals who try to improve their routine find it hard to combine scattered habit data into one clear picture, and they rarely get timely feedback on risks such as too little sleep, excessive screen exposure or growing burnout. Existing tools are either too simple (plain notes with no analysis) or too demanding (sensors, cloud accounts, advertising, unclear data handling). The proposed system provides a lightweight, private, database-backed journal that stores daily records per user and converts them into understandable scores, trends and advice. It supports self-awareness; it does not give medical advice or diagnosis.

2.3 Objectives of the Project

- To provide local account registration, sign-in and sign-out, with validation errors shown clearly and each account's profile and logs kept separate.
- To record one daily entry per date (sleep, exercise, water, screen time, mood, diet, notes) that can be edited, deleted or topped up quickly with 250 ml of water.
- To calculate per-metric scores (0–100), a weighted wellness score and a burnout-risk level (Low, Moderate, Critical) with alerts and recommendations, using transparent rules.
- To present the data through metric cards, 7-day, 14-day, 30-day and all-record trend charts, a mood distribution, a six-axis radar and a 35-day heatmap.
- To offer a built-in advisor that sends nothing anywhere, and an optional, explicitly chosen Gemini advisor for external AI analysis.
- To provide search, filter, CSV export, JSON backup and restore, and a printable report.
- To let the user set personal daily targets and a screen-time ceiling, choose a light or dark theme, and clear the logs when required.
- To store data in a local SQLite database with restricted file permissions and hashed credentials, with no e-mail or cloud dependency.
- To follow a maintainable structure (components, utilities, local API, SQL schema) that can be extended later.

2.4 Scope of the Project

Within scope: local account management; daily journaling; rule-based scoring and burnout analysis; dashboard, charts and history; built-in and optional Gemini advisor; reference library; JSON and CSV export, JSON restore and printable report; daily targets, theme and log purge.

Outside scope: biometric sensors or wearable integration; medically validated diagnosis or treatment advice; e-mail verification, password recovery and account deletion; multi-device synchronisation or cloud hosting; an administrator interface; CSV import; a native mobile application.

2.5 Intended Users

User	Description	Access
Visitor	A person who opens the application without signing in.	Account screen only (sign in or create account).
Registered user	A person who journals and reviews his or her own habits.	Dashboard, daily entry, history, advisor, reference library, settings, backup and report; own data only.
Developer / owner	The person who runs the local service.	Starts the service; reads or backs up the SQLite file in ~/.hnazer (or the folder set in HNAZER_DATA_DIR).

3. PROJECT CATEGORY

The project belongs to the following categories of the BCSP-064 guidelines. It is a client-server web application: a React single-page application talks to a small Node.js service, which stores data in a relational database.

Category	How it applies to this project
RDBMS	Three related tables (accounts, account_data, sessions) with primary keys, a unique username, foreign keys with ON DELETE CASCADE, and persistent local storage in SQLite with write-ahead logging.
Web application / OOPS concepts	React components and state on the client; separate utility modules for scoring, storage, account access and AI; a modular Node.js service on the server; objects model accounts, profiles and daily logs.
Information security	scrypt salted password hashes, random session tokens stored only as SHA-256 hashes with a 30-day expiry, loopback-only binding, owner-only file permissions, token-scoped data access, a request-size limit and no-store caching.
Information management / applied AI	Structured storage and reporting of personal habit data: scores, trends, risk summaries, CSV and JSON exports and a printable report; a deterministic rule-based advisor plus an optional external language-model advisor.

4. ANALYSIS

4.1 Existing System and its Limitations

- Paper journals and notes need manual review, give no score or trend, and are easy to lose.
- Spreadsheets need hand-written formulas, give no instant feedback and are not protected by a login.
- Many health and fitness apps depend on wearables, cloud accounts or e-mail sign-up, show advertisements and keep personal data on third-party servers.
- Few simple tools combine sleep, activity, hydration, screen time, mood and diet into one understandable score.
- Where AI features exist, users often cannot control what data is sent; a built-in option that needs no external service is rare.
- Small personal tools often lack backup, export or a printable summary.

4.2 Proposed System

Hnazer stores one profile and one logbook per account in a local database and calculates all insights from the stored data. The main advantages are:

- Local-first: accounts, profile and logs are kept in SQLite on the user’s own computer, and no e-mail address is needed.
- One daily entry captures six habit areas; saving a date again replaces that date’s entry, so every day has at most one record.
- Transparent scoring: fixed, documented rules give metric scores, a weighted wellness score and a burnout-risk level that the user can inspect.
- Personal targets (sleep, water, exercise and a screen-time ceiling) can be changed in Settings and feed the water, exercise and screen scores.
- The built-in advisor needs no network call; Gemini is used only when the user explicitly chooses it and supplies an API key.
- Every data request carries the session token, and reads and writes are scoped to the account that owns the token.
- Backup, restore, CSV export and a browser-printable report keep the data portable.

4.3 Feasibility Study

Feasibility	Assessment
Technical	All technologies (React, Vite, Tailwind CSS, Chart.js, Node.js, SQLite) are mature, open source and well documented, and are covered by the BCA syllabus and common web-development practice. The built-in node:sqlite module needs Node.js 22.5 or later.
Economic	Development needs only a computer and an internet connection to install packages. SQLite ships inside Node.js, so no paid database server is needed. A Gemini API key is required only if the optional Gemini advisor is used.
Operational	The system runs in a normal web browser and starts with one command (npm run dev). A single dashboard, a simple daily form and a built-in advisor that needs no external service make it easy for non-technical users.

4.4 Software Requirements Specification (SRS)

4.4.1 Purpose and overview

This SRS describes the functions and constraints of Hnazer. The intended readers are the project guide, the evaluator and the developer. The system has one main user class (the registered user), plus visitors and one optional external service (Google's Gemini API). It runs in a modern web browser and depends on a running Node.js service and its SQLite database file on the same computer. It assumes that users enter truthful figures and understand that scores and advice are informational heuristics, not medical diagnoses.

4.4.2 Functional requirements

ID	Module	Requirement
FR-01	Authentication	Create an account with a username, display name and password.
FR-02	Authentication	Validate registration: username trimmed, lower-cased, 3–32 letters, digits or underscores and unique; display name required (up to 80 characters); password of 8–128 characters.
FR-03	Authentication	Store passwords only as salted scrypt hashes; never in browser storage.
FR-04	Authentication	Sign in with username and password and issue a session token valid for 30 days; keep the token in browser local storage.
FR-05	Authentication	Restore the session on later visits; sign out deletes the token on the server and returns to the account screen.
FR-06	Authentication	Create a default profile and an empty log array at registration.
FR-07	Daily entry	Record date, sleep (0–14 h), exercise (0–3 h), water (0–6 L), screen time (0–16 h), mood (six categories), diet (five categories) and optional notes.
FR-08	Daily entry	Keep one log per date: saving the same date replaces the earlier entry; entries can be edited and deleted.
FR-09	Daily entry	Calculate and store the wellness score on save; show a confetti animation at a score of 80 or more.
FR-10	Daily entry	Add 250 ml quickly from the hydration card or adjust the day’s water total; create a default log if the date has none.
FR-11	Analysis	Calculate six metric scores (0–100) from fixed rules and the user’s targets.
FR-12	Analysis	Combine the metric scores into a weighted composite (sleep 25%, exercise 20%, screen 20%, water 15%, mood 10%, diet 10%).
FR-13	Analysis	Calculate a burnout-risk level (Low, Moderate, Critical) from up to the seven most recent records, with alerts and recommendations; return “No Data” when there are no logs.
FR-14	Dashboard	Show the selected date’s dossier and status, vitality overview and six metric cards.
FR-15	Dashboard	Show trend charts (7, 14, 30 days or all records) for sleep, screen time and water, plus mood distribution.
FR-16	Dashboard	Show a six-axis radar and a 35-day heatmap whose cells select the dashboard date.
FR-17	History	List records with score; search by date or notes; filter by mood; edit and delete.
FR-18	History	Export logs as CSV and the profile and logs as a JSON backup.
FR-19	History	Restore a JSON file (raw array or object with a logs array), replacing the account’s logs.
FR-20	AI advisor	Provide a built-in advisor using local rule-based analysis that sends no data out.
FR-21	AI advisor	Provide an optional Gemini advisor, used only on explicit choice, sending the key and a prompt without the notes field.
FR-22	Reference	Provide an in-app reference library (Protocols) on sleep, screen breaks, hydration and stress.
FR-23	Report	Provide a print-formatted report with averages, triage summary, alerts, recommendations and up to 10 recent records.
FR-24	Settings	Edit display name, MRN / user ID, daily targets, screen-time ceiling and Gemini API key.
FR-25	Settings	Purge the current account’s logs after confirmation; keep the account and profile.

ID	Module	Requirement
FR-26	Settings	Provide a light/dark theme switch and save the choice in the profile.
FR-27	Dashboard	Provide a date selector with a Today action.
FR-28	Persistence	Load and save profile and logs through GET and PUT /api/data, scoped to the token’s account.

4.4.3 Non-functional requirements

Attribute	Requirement
Security	Passwords hashed with salted scrypt; session tokens random and stored only as SHA-256 hashes; 30-day expiry; every data call needs a bearer token and is scoped to its account; API and Vite server bound to 127.0.0.1.
Privacy	Data stays on the local machine unless the user explicitly chooses Gemini; the Gemini prompt excludes the notes field; no e-mail address is collected.
Usability	Single dashboard, simple forms, themes, responsive layout (compact controls on small screens).
Performance	A personal logbook is small; the whole profile and log array move in one read and one write, so pages respond almost instantly.
Reliability	API responses are JSON with Cache-Control: no-store; request bodies larger than about 5 MB are rejected; expired sessions are removed when the service starts.
Portability	Runs on Windows, macOS and Linux with Node.js 22.5 or later and a current browser; no database server or native SQLite package is required.
Maintainability	Separate components, utility modules and one service file; scoring rules kept in a single module.
Scalability	The prototype targets one local user or device; moving to a networked deployment would need stronger authentication, rate limiting and a relational log table.

4.5 System Architecture

Hnazer is a client-server application running on one computer. The React client is served by Vite and calls the Node.js service through a small fetch-based account client; Vite forwards every /api request to the service on port 3001. The service checks the bearer token where required, reads or writes SQLite through the built-in node:sqlite module, and returns JSON. Scoring, charts, themes, exports and the built-in advisor run entirely in the browser. When the user chooses Gemini, the browser sends the prompt directly to Google’s API.

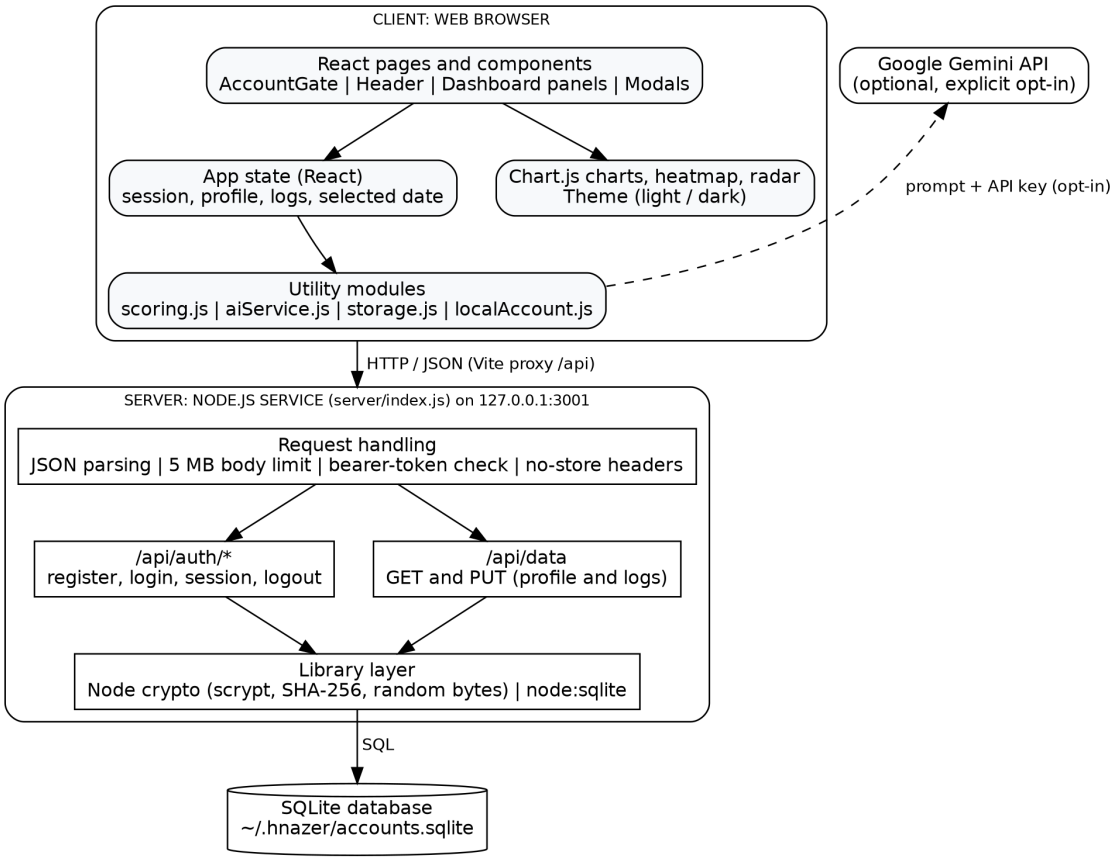


Figure 1: Architecture of Hnazer

Layer	Components	Responsibility
Presentation	React pages and components (AccountGate, Header, VitalityOverview, MetricCards, AnalyticsCharts, BurnoutRadar, HabitHeatmap, QuickWaterLogger, DailyLogModal, HistoryManager, AiAdvisorModal, EducationalHub, ReportModal, SettingsModal).	Renders the interface, validates input and shows results.
State / service	App.jsx state with the localAccount.js client.	Holds session, profile, logs, selected date and theme; calls the API.
Business logic	scoring.js, aiService.js, storage.js.	Scores, risk heuristic, recommendations, advisor, CSV / JSON helpers.
API	Node.js service (server/index.js): /api/auth/* and /api/data.	Authentication, validation, session handling and persistence.
Library	Node crypto (scrypt, SHA-256, random bytes) and node:sqlite.	Password hashing, token hashing and SQL access.
Data	SQLite file accounts.sqlite (default ~/.hnazer).	Persistent storage of accounts, sessions, profile and logs.

4.6 Use Case Diagram

A visitor can create an account or sign in. A registered user adds and reviews daily logs, views the dashboard, scores and burnout risk, uses the advisor and reference library, exports or restores data, prints the report, changes settings and purges logs. The Gemini API takes part only when the optional Gemini advisor is chosen.

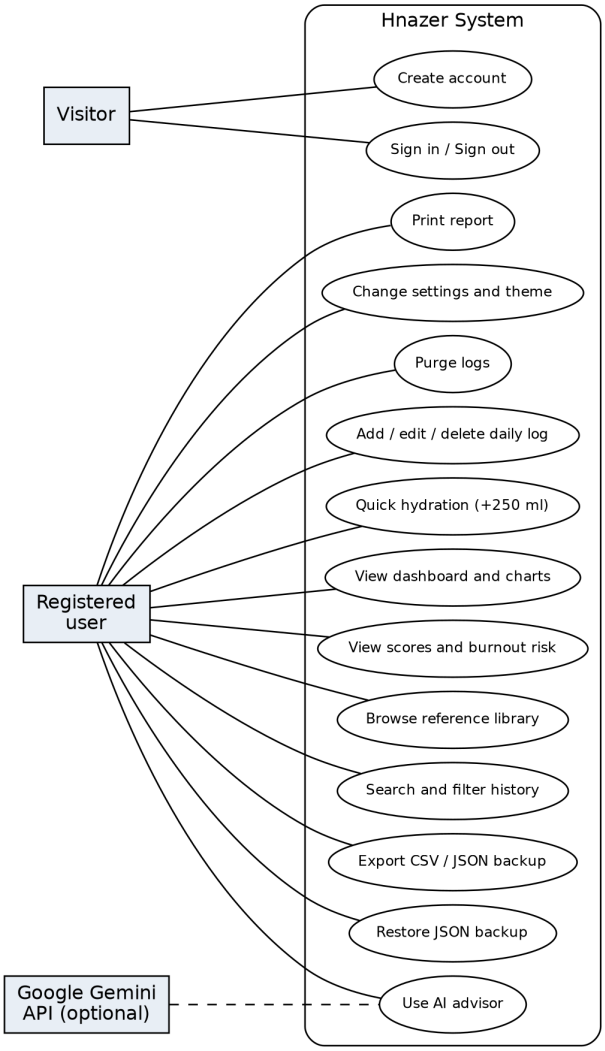


Figure 2: Use case diagram

Actors. Three actors take part. The *Visitor* has not signed in and can only reach the account screen. The *Registered user* has every visitor feature and, in addition, the private features that work on personal habit data. The *Gemini API* is an external system that only receives a prompt when the user explicitly runs the Gemini advisor.

Relationships. Registration and sign-in are the entry points: a successful sign-in opens the dashboard. There is no password-recovery use case in this version because the system has no e-mail address to send a code to.

Pre- and post-conditions.

- *Manage daily log:* the user must be signed in; afterwards the dashboard, charts, heatmap and history are refreshed.
- *View burnout summary:* at least one saved record should exist, otherwise “No Data” and a prompt are shown; the result is calculated fresh from saved records.
- *Use Gemini advisor:* a Gemini API key has been saved and the user explicitly chooses Gemini; the prompt holds the profile name, targets and up to seven recent records without notes.
- *Restore JSON:* the file is a log array or an object with a logs array; the account’s whole log array is replaced, so a backup should be taken first.
- *Purge logs:* the user confirms; the account, password and profile settings remain.

Boundary. The system boundary encloses all use cases; everything outside it (visitors, users and the Gemini API) is an external actor. The table below summarises the main use cases.

Use case	Actor	Description
Create account / Sign in	Visitor	Creates an account or signs in; the service returns a session token and the dashboard opens.
Add / edit / delete daily log	Registered user	Stores one entry per date with its wellness score; edits and deletes later.
Quick hydration	Registered user	Adds 250 ml to the selected date, creating a default log if needed.
View dashboard and scores	Registered user	Shows metric cards, charts, radar, heatmap, wellness score and burnout risk.
Use AI advisor	Registered user, Gemini API	Shows built-in advice, or Gemini advice if explicitly chosen.
Export / back up / restore	Registered user	Downloads CSV or JSON and restores logs from JSON.
Print report	Registered user	Opens a print-formatted summary that the browser can save as PDF.

4.7 Data Flow Diagrams

The data flow diagrams use the Yourdon notation: rectangles are external entities, ellipses are processes and open-ended boxes are data stores. The context diagram is decomposed into a Level 1 diagram and, for four important processes, into Level 2 diagrams.

4.7.1 Context diagram (Level 0)

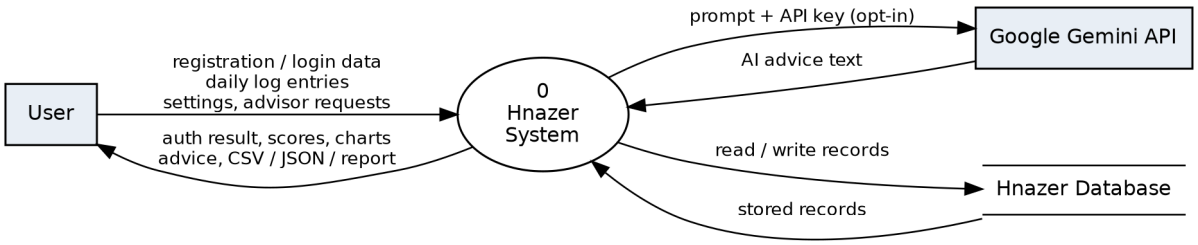


Figure 3: Level 0 DFD (context diagram)

The whole system is a single process. The user supplies registration, login, daily-entry, settings and advisor requests and receives authentication results, scores, charts, advice and CSV, JSON and report outputs. The system reads and writes records in the database and, only on explicit choice, exchanges a prompt and a response with the Gemini API.

4.7.2 Level 1 DFD

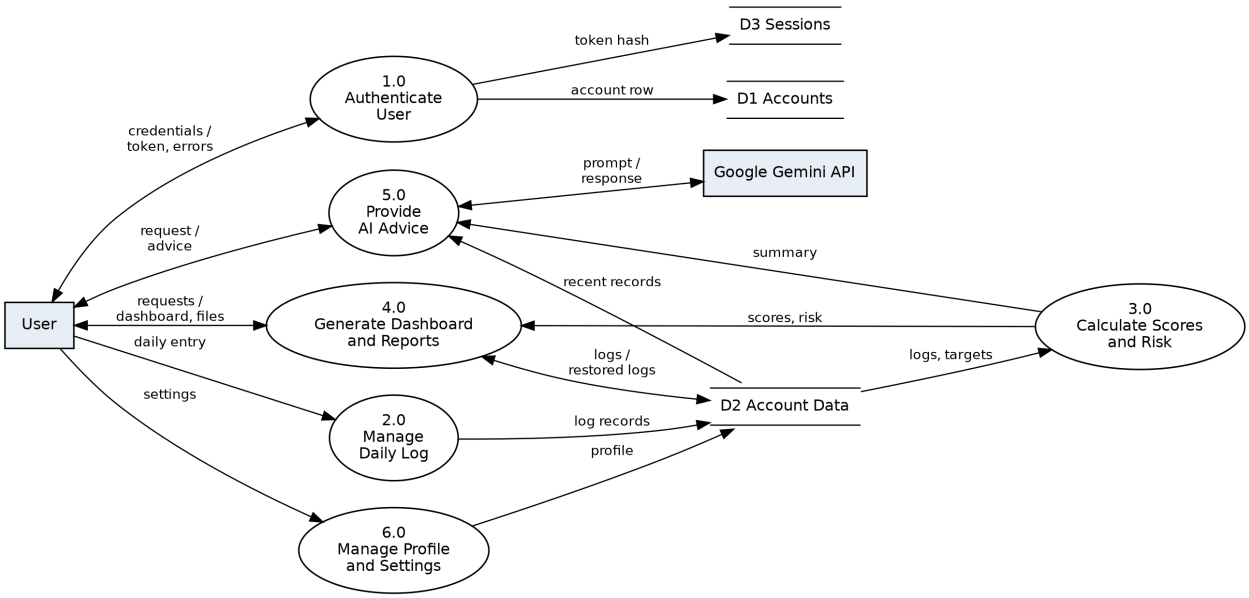


Figure 4: Level 1 DFD

Process	Description
1.0 Authenticate User	Validates credentials, reads or writes D1 Accounts, hashes or verifies the password and creates or removes a session in D3 Sessions.
2.0 Manage Daily Log	Validates an entry, calculates its wellness score and adds, replaces, edits or deletes a record in D2 Account Data; includes quick hydration.
3.0 Calculate Scores and Risk	Reads logs and targets from D2 and computes metric scores, the composite score, burnout risk, alerts and recommendations.
4.0 Generate Dashboard and Reports	Builds cards, charts, radar, heatmap and history; produces CSV, JSON backup and the printable report; restores logs from JSON.
5.0 Provide AI Advice	Runs the built-in advisor or, if chosen, builds a prompt and calls the Gemini API.
6.0 Manage Profile and Settings	Updates display name, targets, theme and API key in the profile; purges logs.

4.7.3 Level 2 DFDs

Process 1.0 – Authenticate User. Input is validated, the account is found or created, the password is hashed (registration) or verified (sign-in) with scrypt, and a session token is created and stored as a hash.

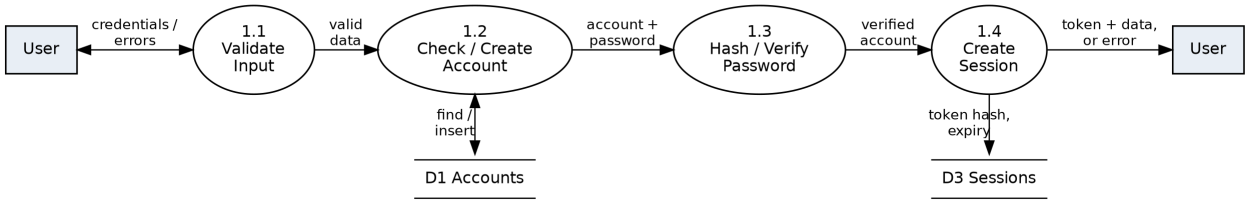


Figure 5: Level 2 DFD for Process 1.0 (Authentication)

Process 2.0 – Manage Daily Log. Every entry is validated, scored and then stored for its date; edits, deletions and quick hydration update the same record list.

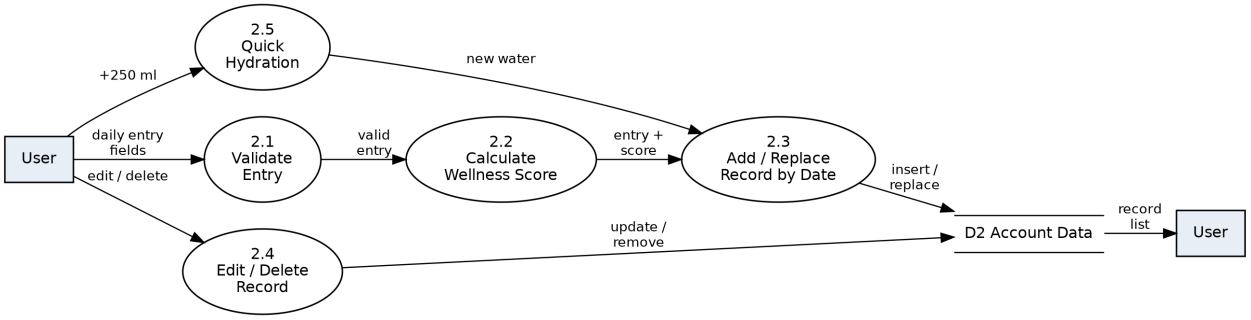


Figure 6: Level 2 DFD for Process 2.0 (Daily log)

Process 3.0 – Calculate Scores and Risk. Records and targets are read, six metric scores and the weighted composite are calculated, and the most recent records are used for the burnout-risk level, alerts and recommendations.

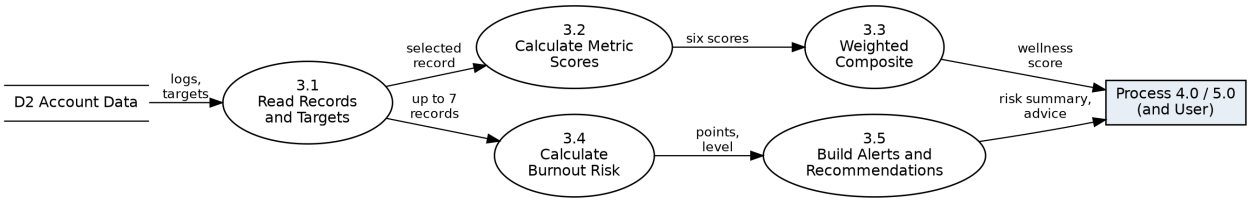


Figure 7: Level 2 DFD for Process 3.0 (Scores and risk)

Process 5.0 – Provide AI Advice. The profile and recent records are read; the built-in analysis runs locally, or, only if chosen, a prompt without notes is sent to Gemini and the reply is shown.

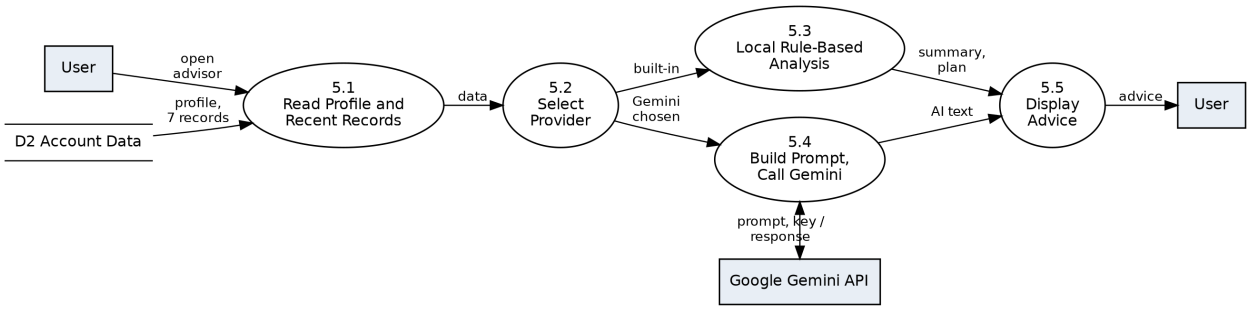


Figure 8: Level 2 DFD for Process 5.0 (AI advisor)

4.8 Entity-Relationship Diagram

The database has three tables. One account has at most one data row (profile and logs) and may have many sessions. Daily logs are a logical entity: they are kept together as a JSON array inside account_data.logs_json rather than as separate relational rows.

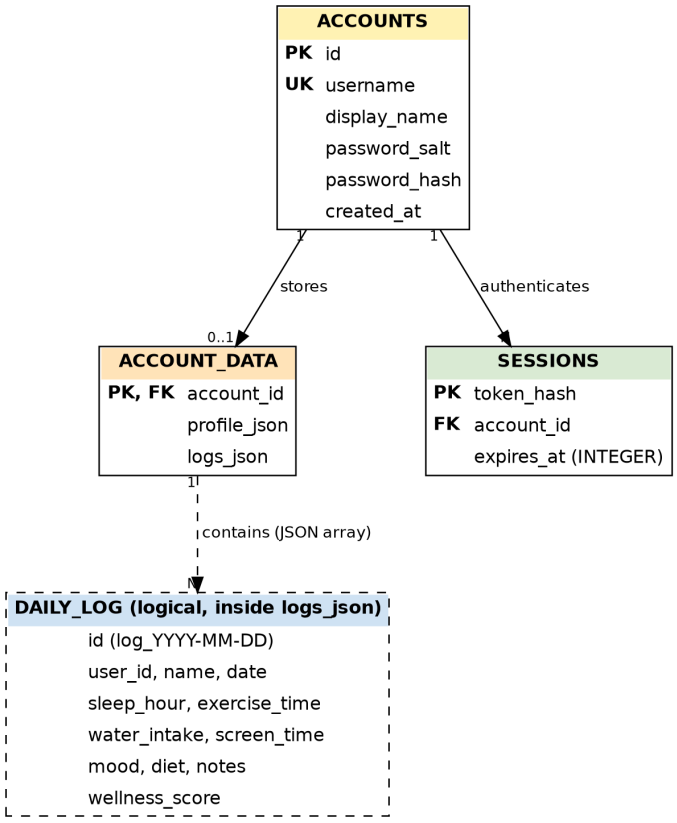


Figure 9: Entity-Relationship diagram

Entities and keys. ACCOUNTS is the parent entity; its id is the primary key and its username is unique. ACCOUNT_DATA uses account_id as both primary key and foreign key. SESSIONS uses the SHA-256 hash of the session token as its primary key and carries account_id as a foreign key.

Relationships and cardinality.

- **stores:** one account to one data row (1 : 0..1); a new account receives its row at registration.
- **authenticates:** one account to many sessions (1 : N); a user may be signed in on several browsers.
- **contains (logical):** one data row holds many daily logs (1 : N) as a JSON array; there is one log per date.

Referential actions. Deleting an account removes its account_data and sessions rows automatically (ON DELETE CASCADE).

Integrity rules. Usernames are unique, 3–32 characters; passwords are stored only as salt and hash; sessions carry an expiry time and expired rows are removed when the service starts; the API checks that a profile object and a logs array are present on every save.

Design note. Keeping the logbook in one JSON document keeps the prototype simple: one read and one write move the whole logbook. The trade-off is that SQL cannot query individual days directly; a later version may move logs into their own table (Section 8).

4.9 Class Diagram

The class diagram shows the main data classes and the services that use them. The App component holds client state; LocalAccountClient and ApiServer handle accounts and persistence; ScoringService and AiService hold the calculation and advisor logic. Operation names are descriptive; the corresponding code is in App.jsx, localAccount.js, server/index.js, scoring.js, aiService.js and storage.js.

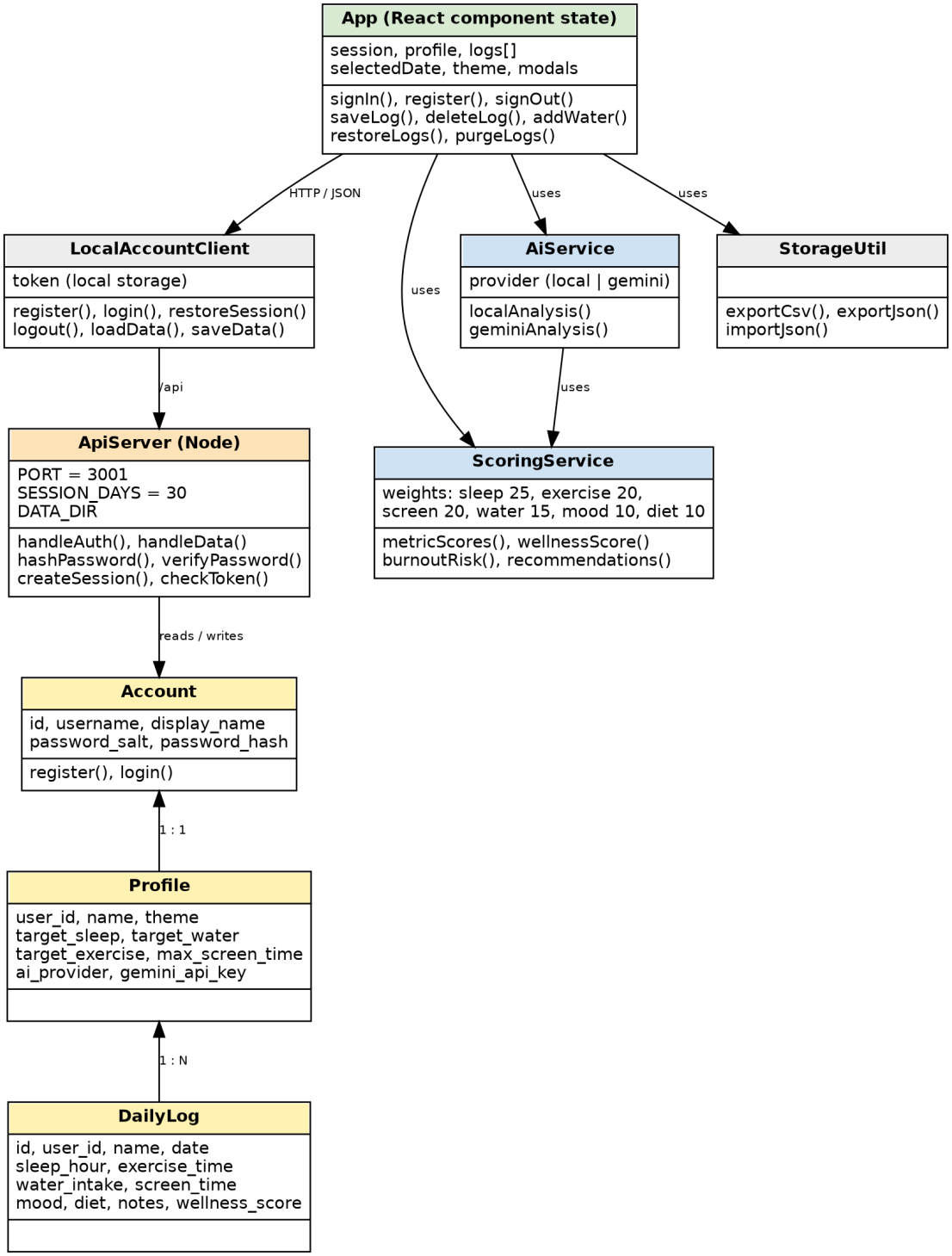


Figure 10: Class diagram

4.10 Data Dictionary

The data dictionary lists the attributes of every table, followed by the two logical JSON objects stored in account_data.

accounts

Field	Type	Required	Constraints and description
id	INTEGER	Auto	Primary key.
username	TEXT	Yes	Login identity; trimmed, lower-case, 3–32 letters, digits or underscores; unique.
display_name	TEXT	Yes	Name shown in the app; required, up to 80 characters.
password_salt	TEXT	Yes	Random salt used for the scrypt hash.
password_hash	TEXT	Yes	scrypt hash of the 8–128 character password; never the plain password.
created_at	TEXT	Yes	Creation timestamp.

account_data

Field	Type	Required	Constraints and description
account_id	INTEGER	Yes	Primary key and foreign key to accounts(id), ON DELETE CASCADE.
profile_json	TEXT	Yes	JSON text of the profile (name, targets, theme, AI settings).
logs_json	TEXT	Yes	JSON text of the array of daily logs.

sessions

Field	Type	Required	Constraints and description
token_hash	TEXT	Yes	Primary key; SHA-256 hash of a random session token (the token itself is not stored).
account_id	INTEGER	Yes	Foreign key to accounts(id), ON DELETE CASCADE.
expires_at	INTEGER	Yes	Expiry time; sessions last 30 days.

daily log (logical object in logs_json)

Property	Type	Description
id	String	Record identifier, generally log_YYYY-MM-DD.
user_id	Number	Profile / MRN identifier.
name	String	Profile display name copied into the record when saved.
date	String	YYYY-MM-DD; used to find or replace the day’s entry.
sleep_hour	Number	Sleep duration in hours (form range 0–14, step 0.5).
exercise_time	Number	Exercise in hours (0–3, step 0.25).
water_intake	Number	Water in litres (0–6, step 0.25).
screen_time	Number	Screen exposure in hours (0–16, step 0.5).
mood	String	Energized, Good, Neutral, Tired, Stressed or Burnout Alert.
diet	String	Healthy, Balanced, Average, Fast Food or Irregular.
notes	String	Optional free text.
wellness_score	Number	Composite score (0–100) calculated when the entry is saved.

profile (logical object in profile_json)

Property	Default	Description
user_id	Account ID	MRN / user identifier shown in the app.
name	Registration name	Display name.
target_sleep	8 hours	Sleep target (UI range 4–12 h, step 0.5).
target_water	3 litres	Water target (1–8 L, step 0.25).
target_exercise	0.75 hours	Exercise target (0.25–4 h, step 0.25).
max_screen_time	5 hours	Screen-time ceiling (2–14 h, step 0.5).
theme	dark	Light or dark theme.
ai_provider	Built-in	Chosen advisor provider.
gemini_api_key	Empty	Optional Gemini API key.
created_at	Creation time	Profile creation timestamp.

5. COMPLETE STRUCTURE OF THE PROJECT

5.1 Modules and their Description

The project is divided into eight modules. The scoring engine, dashboard and persistence modules need the most effort because they combine calculations, charts, validation and SQL.

Module	Description	Effort
M1 Navigation, Theme and Settings	Header with brand, date selector and Today action, theme toggle and sign-out; Settings modal (name, MRN, targets, screen ceiling, Gemini key, purge logs).	Low
M2 Authentication and Session	Account screen (create account / sign in), scrypt hashing, session token, session restore and sign-out, local account client.	Medium
M3 Daily Journal Entry	Daily entry form, one record per date, edit and delete, quick +250 ml, water logger.	Medium
M4 Scoring and Analysis Engine	Metric scores, weighted wellness score, burnout heuristic, alerts and recommendations (scoring.js).	High
M5 Dashboard and Visualisation	Dossier and status, vitality overview, metric cards, trend and mood charts, six-axis radar, 35-day heatmap.	High
M6 History, Backup and Reports	History search and filter, CSV export, JSON backup and restore, print report.	Medium
M7 AI Advisor and Reference Library	Built-in advisor, optional Gemini advisor, Protocols reference library.	Medium
M8 Persistence and API Layer	Node.js routes, SQLite schema, validation, error handling, file permissions.	High

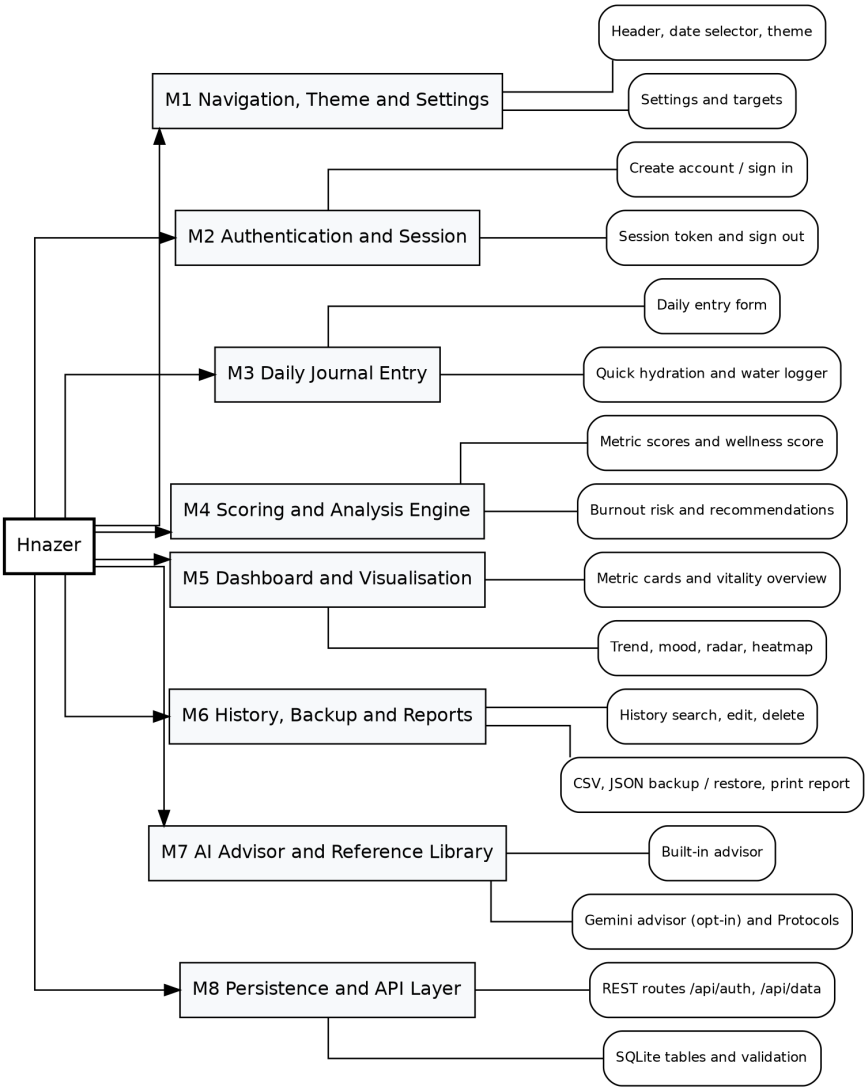


Figure 11: Module hierarchy

5.2 Data Structures

Structure	Kind	Purpose
Account row	{ id, username, display_name, password_salt, password_hash, created_at }	Account identity stored in SQLite.
Profile object	{ user_id, name, target_sleep, target_water, target_exercise, max_screen_time, theme, ai_provider, gemini_api_key, created_at }	Personal settings and targets.
Daily log	{ id, user_id, name, date, sleep_hour, exercise_time, water_intake, screen_time, mood, diet, notes, wellness_score }	One day's journal entry.
Metric scores	Six numbers 0–100 (sleep, water, exercise, screen, mood, diet)	Input to the composite score and radar.
Risk result	{ points, level, averages, alerts, recommendations }	Burnout summary; "No Data" when no logs exist.
Session token	Random string in browser; SHA-256 hash in sessions table	Identifies the account in every protected request.
Chart datasets	Arrays of dates and values (7, 14, 30 days or all)	Feed the trend and mood charts.
Heatmap grid	Last 35 dates with per-record scores	Colour-coded activity overview.
Export formats	CSV columns user_id, name, date, sleep_hour, exercise_time_hours, water_intake_litres, screen_time_hours, mood, diet, notes; JSON backup with app name, version, time, profile and logs	Portable copies of the data.
React state objects	session, profile, logs[], selectedDate, theme, open modals	Application state kept in the App component.

5.3 Database Design

The database file is created by the Node.js service on first use, in ~/.hnazer/accounts.sqlite by default or in the folder named by HNAZER_DATA_DIR. The folder is created with owner-only permissions (0700) and the file with owner read/write permissions (0600). SQLite write-ahead logging is enabled, and the schema is applied automatically when the service starts.

Integrity and constraints

- accounts.username is unique; passwords are stored only as salt and scrypt hash.
- account_data and sessions reference accounts(id) and are deleted automatically with the account (ON DELETE CASCADE).
- Each account has exactly one account_data row, created at registration, holding the profile and the log array as JSON text.
- Sessions store only the SHA-256 hash of the token and an expiry time; expired sessions are removed when the service starts.
- The database contents are not encrypted at rest; protection relies on the operating-system account and file permissions.

Table definitions (excerpt, simplified)

```
CREATE TABLE accounts (id INTEGER PRIMARY KEY, username TEXT NOT NULL UNIQUE,
  display_name TEXT NOT NULL, password_salt TEXT NOT NULL, password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL);

CREATE TABLE account_data (
  account_id INTEGER PRIMARY KEY REFERENCES accounts(id) ON DELETE CASCADE,
  profile_json TEXT NOT NULL, logs_json TEXT NOT NULL);

CREATE TABLE sessions (token_hash TEXT PRIMARY KEY,
  account_id INTEGER NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  expires_at INTEGER NOT NULL);
```

5.4 Process Logic for Each Module

The logic of each module is given below. Figure 12 shows how the screens are connected, Figure 13 the scoring and burnout-risk process and Figure 14 the overall workflow.

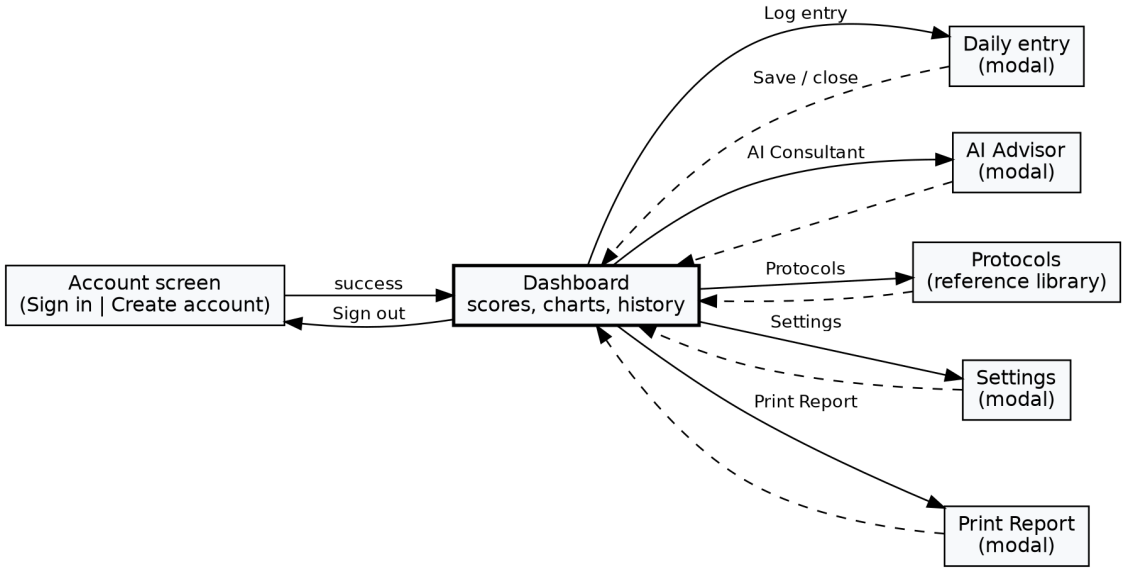


Figure 12: Page navigation of the system

M2 Authentication

1. Receive display name, username and password; the form checks the rules and shows errors on the account screen.
2. The service validates again, checks that the username is new (registration) or finds the account (sign-in).
3. Hash the password with scrypt and a random salt (registration) or compare it with the stored hash (sign-in).
4. Create a random session token valid for 30 days, store only its SHA-256 hash, and return the token with the account, profile and logs.
5. The client keeps the token in local storage, loads the dashboard and, on later visits, restores the session; sign-out deletes the session on the server.

M3 Daily Journal Entry

1. Open the entry form with the selected date and enter sleep, exercise, water, screen time, mood, diet and notes within the form ranges.
2. Calculate the wellness score and save the record under its date, replacing any earlier record for that date.
3. Refresh dashboard, charts, heatmap and history; show confetti if the score is 80 or more.
4. Quick hydration adds 0.25 L to the selected date (creating a default log if needed); records can be edited or deleted from the history.

M4 Scoring and Analysis Engine

The calculations are fixed heuristics implemented in one module; they are not a clinically validated model. Each component score is between 0 and 100.

Metric	Scoring rule
Sleep	7–9 h: 100; 6 to under 7 h: 80; over 9 to 10 h: 85; 5 to under 6 h: 60; 4 to under 5 h: 35; under 4 h: 15; over 10 h: 65.
Water	water_intake / target_water × 100, rounded and capped at 100.
Exercise	exercise_time / target_exercise × 100, rounded and capped at 100.
Screen	Up to 3.5 h: 100; up to the configured maximum: 85; up to maximum + 2 h: 60; up to maximum + 4 h: 35; higher: 15.
Mood	Energized 100; Good 85; Neutral 70; Tired 45; Stressed 35; Burnout Alert 15.
Diet	Healthy 100; Balanced 85; Average 65; Fast Food 40; Irregular 30.

Weighted composite: sleep 25% + exercise 20% + screen 20% + water 15% + mood 10% + diet 10%, rounded and limited to 0–100. The sleep target affects the progress display only; the sleep score itself uses the fixed bands above.

Recent burnout heuristic. Up to the seven most recent saved records are used (not necessarily seven consecutive dates). Points are added for each condition met:

Condition over recent records	Points
Average sleep below 6 hours	30
Average screen time above 7 hours	25
Average water below 2 litres	15
Average exercise below 0.3 hours (18 minutes)	20
At least 3 records marked Tired, Stressed or Burnout Alert	20

The total is capped at 100: under 25 points is Low, 25–49 is Moderate and 50 or more is Critical. Text alerts and recommendations are created for the triggered conditions. With no logs the engine returns “No Data” and a prompt to create an entry.

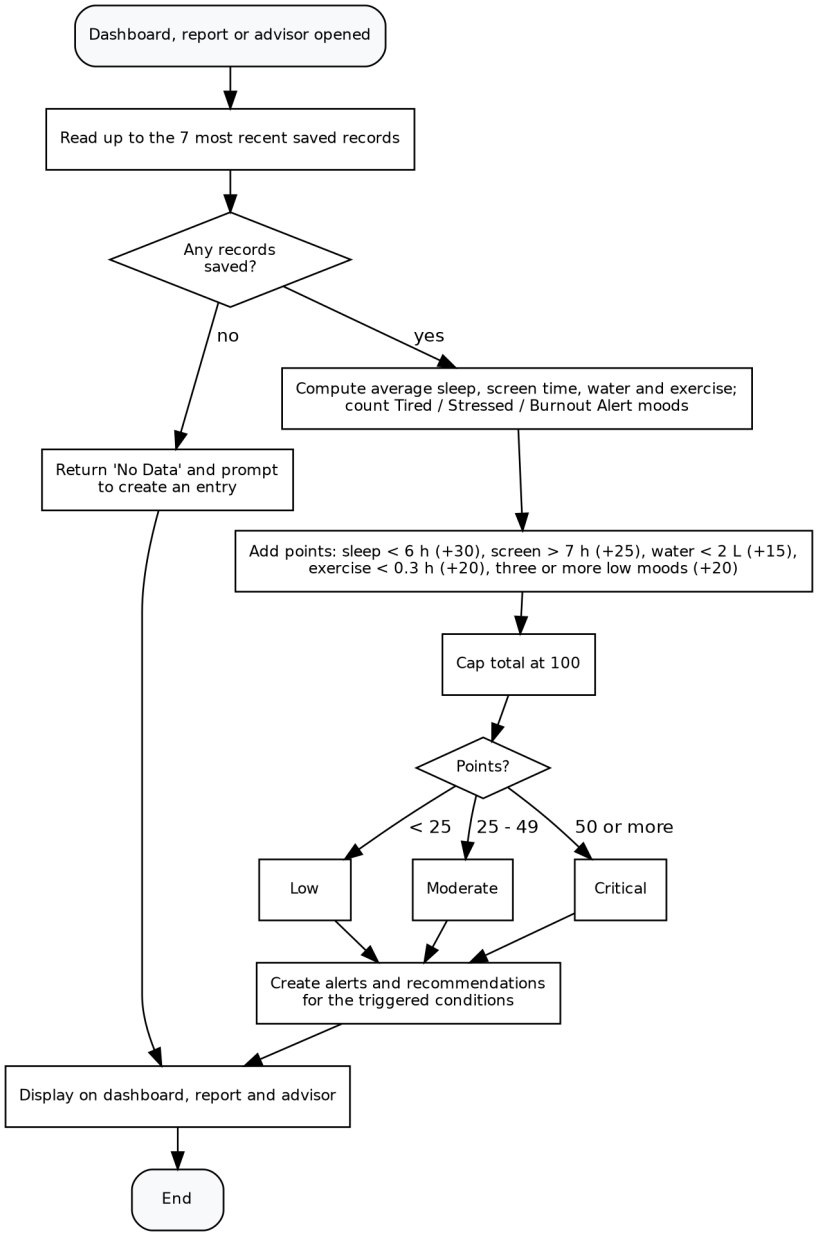


Figure 13: Scoring and burnout-risk process

M5 Dashboard and Visualisation

1. Load profile and logs after sign-in; show the selected date's dossier, status and vitality overview.
2. Show six metric cards for the selected date, with the +250 ml hydration action.
3. Draw trend charts for sleep, screen time and water over 7, 14, 30 days or all records, plus the mood distribution (the water line is scaled to share the axis with hour-based lines, as stated in the legend).
4. Draw the six-axis radar from the metric scores and the 35-day heatmap; selecting a heatmap day changes the dashboard date.

M6 History, Backup and Reports

1. List records with their score; search by date or notes and filter by mood; edit or delete a record.
2. Export CSV (logs only) or download a JSON backup (app name, version, time, profile and logs).
3. Restore JSON replaces the account's whole log array; profile and targets are not restored from the file.
4. Print Report opens a summary with averages, triage, alerts, recommendations and up to 10 recent records; the browser's print dialog can save it as PDF.

M7 AI Advisor and Reference Library

1. The built-in advisor derives a recent-record summary and a fixed action plan using local rule-based analysis; it uses no machine-learning model and sends nothing out.
2. The Gemini advisor runs only when explicitly chosen: the browser sends the API key and a prompt (profile name, targets and up to seven recent records without notes) to Google’s Generative Language API and shows the reply.
3. The Protocols library shows reference text on sleep, screen breaks, hydration and stress management.

M8 Persistence and API Layer

1. POST /api/auth/register and /api/auth/login validate input and return the account, profile, logs and session token.
2. GET /api/auth/session restores a current session; POST /api/auth/logout deletes it.
3. GET /api/data returns the token holder’s profile and logs; PUT /api/data replaces them and requires a profile object and a logs array.
4. All responses are JSON with Cache-Control: no-store; bodies over about 5 MB are rejected; data access is scoped to the account of the bearer token.

Overall workflow

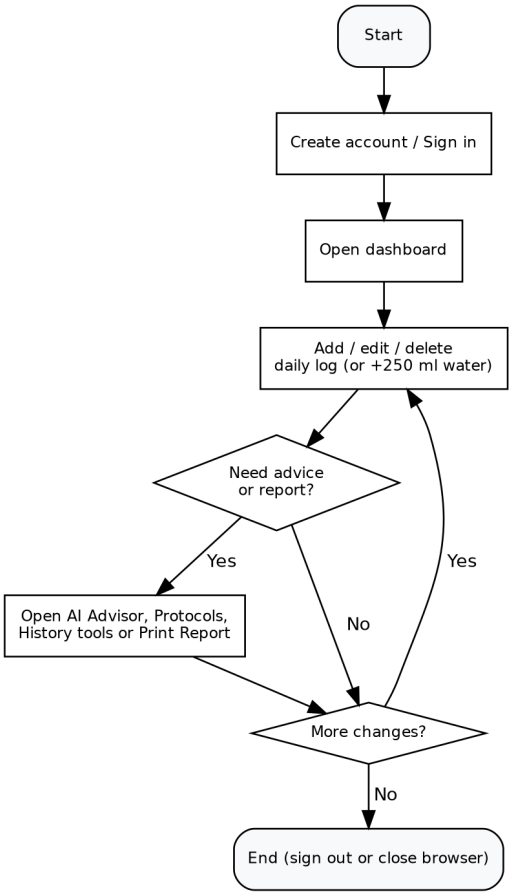


Figure 14: Overall workflow of the system

Walkthrough. A user starts by creating an account or signing in. The dashboard opens and the user adds, edits or deletes daily logs; each change immediately updates the metric cards, charts, radar, heatmap and history.

Decision 1 – Need advice or report? If the user wants advice, the AI Advisor, Protocols, history tools or Print Report can be opened from the dashboard at any point; they only read the saved data, so the user can close them and continue.

Decision 2 – More changes? If more entries are needed, the flow returns to the daily log; otherwise the session ends, either by signing out or by closing the browser. The session token stays valid for 30 days unless the user signs out.

5.5 Testing Details

Testing follows the levels required by the guidelines. Test cases are designed for each level, results are recorded in the project report, and defects found are logged, debugged and corrected. The current package has no test or lint script, so the automated tests below are planned work; the production build is the present compile check.

Level	Focus	Approach
Unit testing	Scoring rules, weighted composite, burnout heuristic, input validation, CSV and JSON helpers.	Vitest with sample inputs (planned).
Integration testing	API routes with SQLite; token and account scoping; session expiry and logout.	Route-level tests against a temporary HNAZER_DATA_DIR.
System testing	Complete journeys in the browser: sign-up, daily entry, dashboard, history, backup and restore, advisor, settings.	Manual test scripts and browser automation.

Level	Focus	Approach
Acceptance testing	Clarity of the dashboard, scores and advice with sample users.	Guided trials with test data.

Sample test cases

ID	Module	Test case	Expected result
TC-01	Auth	Create an account with a valid display name, username and 8+ character password.	Account created; dashboard opens with an empty logbook.
TC-02	Auth	Register a username that already exists.	Error shown; no duplicate account.
TC-03	Auth	Use the username “ab”, or one with spaces or symbols.	Validation error; account not created.
TC-04	Auth	Use a password of 7 characters.	Password-length error; no account.
TC-05	Auth	Sign in with a wrong password.	Error shown; no session created.
TC-06	Auth	Reload the page with a valid token.	Session restored; profile and logs load.
TC-07	Auth	Sign out, then call GET /api/data with the old token.	401 response.
TC-08	Security	Call GET /api/data without a token.	401 response.
TC-09	Security	Sign in as two accounts and load data for each.	Each token returns only its own profile and logs.
TC-10	Entry	Save sleep 8 h, exercise 0.75 h, water 3 L, screen 3 h, Energized, Healthy (default targets).	Wellness score 100; confetti animation.
TC-11	Entry	Save a second entry for the same date.	Earlier entry replaced; one record for that date.
TC-12	Score	Save sleep of 5.5 h.	Sleep score 60.
TC-13	Score	Sleep 6.5 h, exercise 0.375 h, screen 6 h, water 1.5 L, Tired, Average (default targets).	Metric scores 80, 50, 60, 50, 45, 65 (in the order sleep, exercise, screen, water, mood, diet); wellness score 61.
TC-14	Risk	Three records with sleep 5 h, screen 8 h, water 1.5 L, exercise 0.1 h and mood Stressed.	30 + 25 + 15 + 20 + 20 = 110, capped at 100; level Critical.
TC-15	Risk	Records where only the average water is below 2 L.	15 points; level Low.
TC-16	Risk	Open the dashboard with no logs.	“No Data” and a prompt to create an entry.
TC-17	Hydration	Press +250 ml on a date with no log.	New record with default values and water 0.25 L.
TC-18	History	Edit one record and delete another; reload.	Changes persist.
TC-19	Export	Export CSV with logs, and with no logs.	File with the specified columns; no action when empty.
TC-20	Backup	Download JSON backup, purge logs, restore the file.	Logs return; profile unchanged.
TC-21	AI	Run the built-in advisor.	Local analysis shown; no Gemini request made.
TC-22	AI	Run the Gemini advisor with an invalid key.	Error shown; the application stays usable.
TC-23	Settings	Change the water target to 2 L and save.	Water score recalculated against 2 L; setting persists.
TC-24	Settings	Toggle the theme, sign out and sign in.	Theme saved with the profile and restored.

ID	Module	Test case	Expected result
TC-25	Security	Send a request body larger than about 5 MB.	Request rejected.

5.6 Reports Generation

Hnazer produces the following outputs for the signed-in user.

Output	Input	Result
Vitality overview	Selected log and recent records	Wellness score, burnout summary, recommendations and record counts.
Metric cards	Selected date's log	Sleep, exercise, water, screen, mood and diet summaries with quick +250 ml.
Trend charts	Saved records	Lines for sleep, screen time and water over 7, 14, 30 days or all records, plus mood distribution.
Six-axis radar	Metric scores	Polygon of the six scores.
35-day heatmap	Per-record scores	Colour-coded grid; selecting a day changes the dashboard date.
History table	Saved records	Searchable, filterable list with edit and delete.
CSV export	Logs	Columns user_id, name, date, sleep_hour, exercise_time_hours, water_intake_litres, screen_time_hours, mood, diet, notes.
JSON backup	Profile and logs	File with app name, version, export time, profile and logs.
Printable report	Averages, triage, alerts, up to 10 recent records	Print-formatted summary; the browser can save it as PDF.
Advisor output	Profile and recent records	Built-in or Gemini analysis text.

Tentative contents of the project report

1. Introduction and objectives; tools and environment used.
2. Analysis document: SRS, E-R diagram, data flow diagrams, class diagram, data dictionary.
3. Design document: modularisation, database design and integrity constraints, procedural design, user-interface design.
4. Program code with comments and description.
5. Testing: test-case design and outcome reports for unit, integration and system testing; debugging and code improvement.
6. Input and output screens.
7. Implementation of security.
8. Limitations, future scope and bibliography.

5.7 Security Implementation

- Passwords are hashed with a random salt using scrypt before storage; the plain password is never stored in the database or in browser storage.
- Session tokens are random, last 30 days and are stored in the database only as SHA-256 hashes; sign-out deletes the session and expired sessions are removed when the service starts.
- Every data call carries the bearer token, and reads and writes are scoped to the account that owns it.
- The API and the Vite development server bind to 127.0.0.1; the data folder has owner-only permissions (0700) and the database file 0600.

- API responses use Cache-Control: no-store and request bodies larger than about 5 MB are rejected.
- The built-in advisor sends nothing out; Gemini is opt-in, and its prompt omits the notes field.
- Known gaps are stated openly: the database and exports are not encrypted, the Gemini key is saved in the profile and appears in JSON backups, and there is no rate limiting. The prototype is meant for one local user and must not be exposed directly to a network.

6. TOOLS / PLATFORM, HARDWARE AND SOFTWARE REQUIREMENTS

6.1 Tools and Technologies

Layer	Technology	Purpose
Front end	React 18, Vite	User interface and development server.
Styling	Tailwind CSS, PostCSS, Autoprefixer	Responsive layout and light/dark themes.
Charts	Chart.js	Trend and mood charts and radar.
Back end	Node.js 22.5 or later (server/index.js)	Local JSON API, authentication and Vite launcher.
Security	Node crypto (scrypt, SHA-256, random bytes)	Password hashing, token hashing, secure random tokens.
Database	SQLite through node:sqlite	Relational storage with SQL; no server installation.
AI	Built-in rule-based advisor; Google Gemini API (optional)	Local analysis; optional external AI analysis.
Utilities	npm; Google Fonts (remote)	Package management and fonts.

6.2 Software Requirements

- Operating system: Windows, macOS or Linux capable of running Node.js.
- Runtime: Node.js 22.5 or newer with npm.
- Browser: a current version of Chrome, Edge or Firefox with JavaScript and local storage enabled.
- AI: a Gemini API key (optional; without it only the built-in advisor is used).
- Optional: DB Browser for SQLite to view the database file.
- Commands: npm ci, npm run dev, npm run build.

6.3 Hardware Requirements

Profile	Recommended specification
Minimum (development)	Dual-core 2 GHz processor, 4 GB RAM, 2 GB free storage, internet connection.
Recommended (development)	Four-core processor, 8 GB RAM, SSD storage, broadband internet.
Client (runtime)	Any laptop or desktop that can run Node.js and a modern browser.

The specifications are estimates and depend on the amount of data.

7. INDUSTRY / CLIENT

Are you doing this project for any Industry / Client? **No.**

The project is an individual academic work formulated with the help of the project guide. It is not developed for any industry or client.

8. FUTURE SCOPE AND FURTHER ENHANCEMENT

8.1 Limitations of the Present Version

- There is no password recovery, e-mail verification, account-deletion screen or multi-device synchronisation.
- The SQLite database and the CSV / JSON exports are not encrypted; the Gemini key is saved in the profile and is included in JSON backups.
- All values are entered by the user; scores and advice are fixed heuristics, not clinically validated, and are not medical diagnoses.
- The sleep score uses fixed bands and ignores the sleep target; streak helpers count saved records, not strictly consecutive days.
- The whole logbook is stored as one JSON document per account, so individual days cannot be queried with SQL.
- The service is for one local user on a loopback address; there is no rate limiting or production session hardening.
- The production build is frontend-only; the account service must be started with npm run dev.
- The printable report needs at least one saved record, and PDF output depends on the browser's print dialog.
- The package defines no automated test or lint script yet.

8.2 Future Scope

- Password recovery with e-mail or OTP verification, account deletion and two-factor authentication.
- Encryption at rest, HTTP-only cookie sessions, rate limiting and deployment with HTTPS.
- A separate relational table for daily logs, enabling SQL queries and long-term reports.
- Wearable and sensor integration for measured sleep, activity and heart-rate data.
- Editable score weights and personalised thresholds; validated models in place of fixed heuristics.
- Direct server-side PDF reports, CSV import and consecutive-day streaks.
- Automated unit, integration and end-to-end tests with continuous integration.
- An installable progressive web app or mobile application with optional synchronisation.

9. BIBLIOGRAPHY

1. R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
2. I. Sommerville, *Software Engineering*, Pearson Education.
3. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, Pearson Education.
4. React Documentation – https://react.dev
5. Vite Documentation – https://vite.dev/guide/
6. Tailwind CSS Documentation – https://tailwindcss.com/docs
7. Chart.js Documentation – https://www.chartjs.org/docs/
8. Node.js Documentation (node:sqlite and crypto modules) – https://nodejs.org/api/
9. SQLite Documentation – https://www.sqlite.org/docs.html
10. Google Gemini API Documentation – https://ai.google.dev/gemini-api/docs
11. OWASP Cheat Sheet Series: Password Storage and Session Management – https://cheatsheetseries.owasp.org
12. IGNOU School of Computer and Information Sciences, BCSP-064 Project Guidelines.

10. DECLARATION

I declare that this synopsis is my own original work, prepared in consultation with my project guide, and that it has not been copied from any other source or submitted to any other university or institute.

Signature of the Student	**Signature of the Guide**
Name: ……	Name: ……
Enrolment No.: ……	Designation: ……
Date: …….	Date: …….